SISTEMA DE RECUPERACIÓN Y COMPRENSIÓN DE NORMATIVA SENASA

Grupo II:
- Fica Millán, Yesica
- Petraroia, Franco

TFI · Seminario de PLN · 2026

TF-IDF | Embeddings | BERT QA | FastAPI | FAISS

Pipeline end-to-end:
recuperación de información + pregunta-respuesta sobre resoluciones oficiales del SENASA.

Enfoques comparados:

- Modelo clásico: TF-IDF + similitud coseno
- Modelo profundo: Embeddings + BERT QA
- Modelo híbrido: interpolación $\alpha=0.8$

El Problema: Normativa difícil de consultar

Contexto y propuesta de valor

01

Volumen

SENASA genera cientos de resoluciones anuales en PDF, con texto técnico denso.

02

Búsqueda manual

Profesionales leen decenas de páginas para un artículo. Lento y propenso a errores.

03

Sin trazabilidad

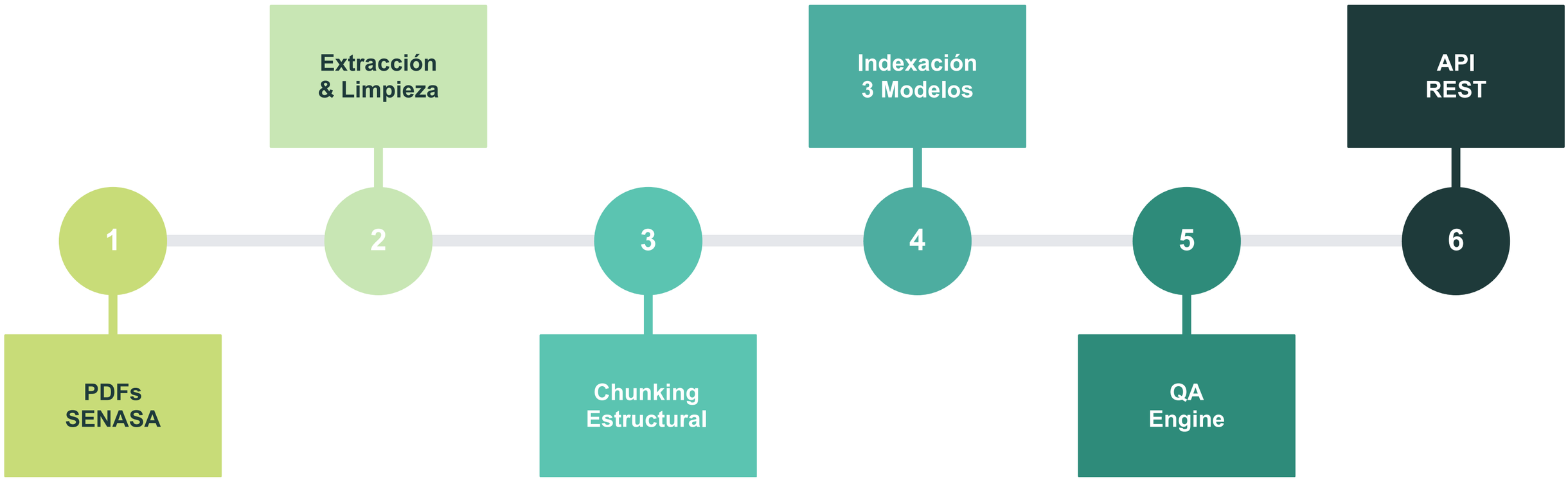
No existe sistema que indique exactamente en qué resolución y artículo está la respuesta.

Nuestra propuesta

- Corpus: 10 resoluciones SENASA (2002–2025) · dominio sanidad animal
- Consultas en lenguaje natural con respuesta trazable al documento fuente
- Comparación TF-IDF (clásico) vs. Embeddings + BERT (profundo) · API REST

Arquitectura: Pipeline End-to-End

De los PDFs a la respuesta en lenguaje natural



Decisiones clave de diseño:

- Chunking estructural (no es ventana fija)
- Singleton pipeline: modelos cargados 1 vez
- Fallback controlado: degrada sin romperse

Chunking Estructural: ¿Por qué no ventana fija?

✗ Ventana fija

✗ Corta artículos a la mitad

✗ Chunks sin coherencia semántica

✗ No distingue VISTO / ARTÍCULO / CONSIDERANDO

✗ Pierde el contexto jurídico de las cláusulas 'Que...'

Chunking estructural - Metadatos

✓ doc_id, chunk_id → trazabilidad hacia el documento fuente

✓ tipo_seccion → VISTO / CONSIDERANDO / ARTICULO / ANEXO

✓ CONSIDERANDO se subdivide por cláusulas 'Que...'

✓ Cada artículo es una unidad semántica preservada

✓ article_number → activa la regla de bypass estructural
si query dice 'artículo 3' → score=1.0 directo

✓ detección de intención de la query mediante uso de flags estructurales en los chunks flags semánticos: es_definicion, es_obligacion, es_procedimiento, es_sancion

Resultado: 558 chunks | Tamaños: min=15 max=120 prom=79.9 | sin duplicados | 95 secciones distintas

Modelo Clásico vs. Modelo Semántico

TF

TF-IDF - Modelo Clásico

Coincidencia de palabras exactas

- Cuenta cuántas veces aparecen los términos de la query en cada chunk
- Muy preciso con vocabulario técnico repetitivo (SENASA)
- Sin GPU · sin descarga de modelos · determinístico
- Limitación: no entiende sinónimos ni paráfrasis

EM

Embeddings - Modelo Semántico

Comprensión del significado

- Convierte texto en vectores de 384 dim · textos similares quedan cerca
- Entiende '¿qué regula la norma?' aunque no repita esas palabras
- MiniLM-L12-v2 + FAISS IndexFlatIP · búsqueda exacta
- Limitación: entrenado en corpus general (zero-shot sobre SENASA)

Cada modelo tiene ventajas según el tipo de consulta → la combinación es la solución

Modelo Híbrido: Lo mejor de los dos enfoques

$$score_hybrid = \alpha \times score_embeddings + (1 - \alpha) \times score_tfidf \quad \text{con } \alpha = 0.8$$

¿Por qué α = 0.8?

- α optimizado con grid search sobre MRR
- α=0.8 → 80% embeddings, 20% TF-IDF
- Consultas reales son paráfrasis, no términos exactos
- TF-IDF evita pérdida de precisión en términos técnicos específicos

Normalización de scores

- TF-IDF y embeddings tienen escalas distintas
- Normalizamos ambos al rango [0, 1] antes de combinar
- Sin normalización, un score dominaría al otro sin importar α
- La interpolación lineal es semánticamente válida

TF-IDF MRR = 0.484	Embeddings MRR = 0.397	Híbrido ✓ MRR = 0.561

QA - Question Answering

de chunks recuperados a respuesta en lenguaje natural

01 Extractivo (baseline)

- Sin modelo neuronal
- Selecciona oración con mayor overlap léxico con la query
- Rápido y determinístico
- Establece el piso de comparación

02 Sintético (heurístico)

- Combina fragmentos de múltiples chunks
- Mayor cobertura temática
- Mejor F1 en modo híbrido (F1=0.250)
- Riesgo: puede mezclar contextos distintos

03 Transformer (BERT)

- BERT fine-tuneado en SQuAD 2.0 español
- Predice span exacto (inicio y fin) en el contexto
- No genera texto nuevo: extrae un span del contexto
- La calidad del QA depende críticamente del retriever (pipeline RAG)

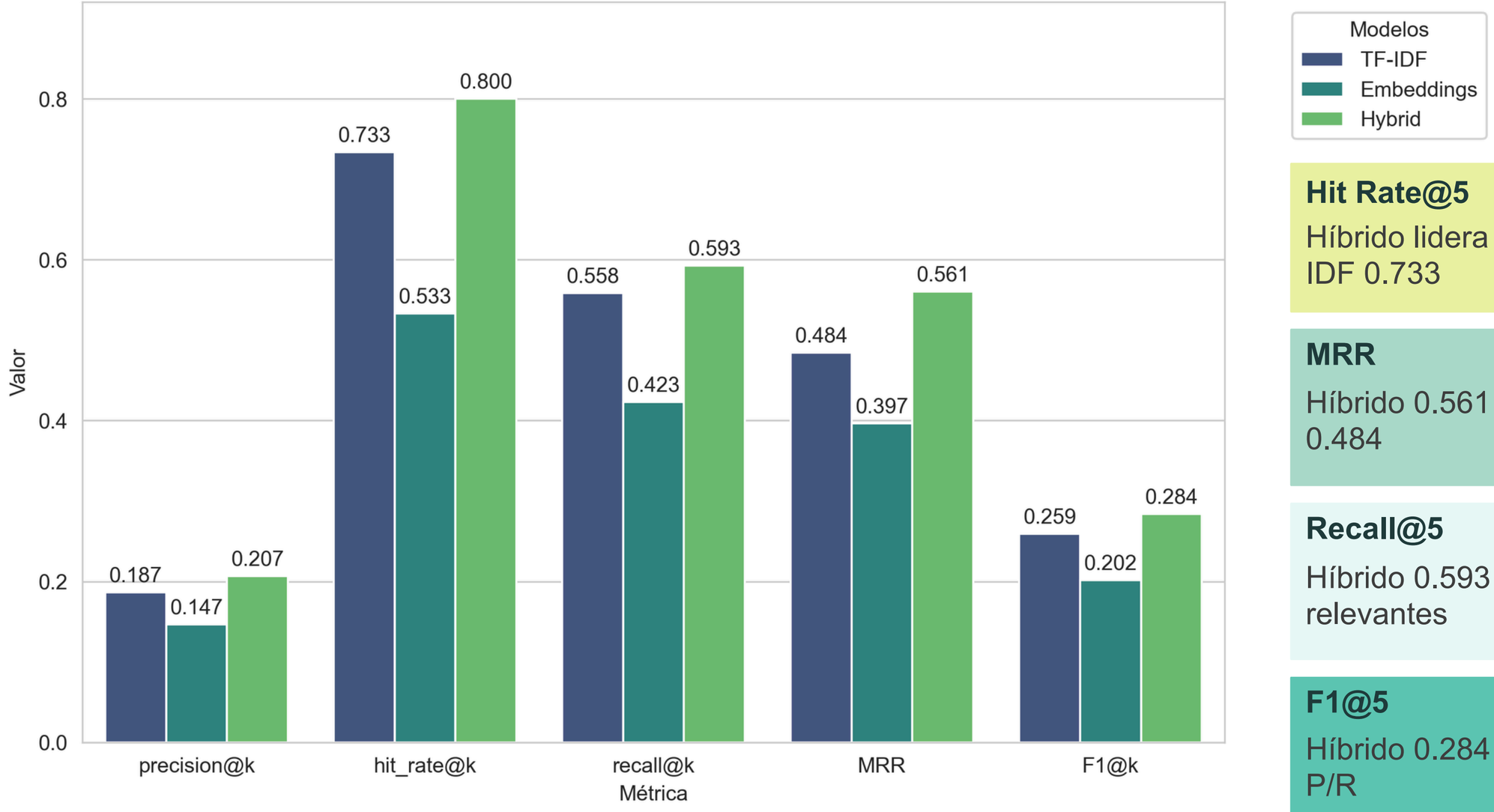
04 Transformer (DistilBERT)

- Modelo más liviano (DistilBERT) fine-tuneado en SQuAD 2.0
- Concatena top-5 chunks en un solo contexto (1 inferencia)
- Trade-off: más eficiente pero la concatenación introduce ruido semántico y dificulta la extracción del span

⚠ EM = 0 en todos los casos: es esperable. El EM exige igualdad exacta carácter a carácter.
En QA sobre texto libre la métrica válida es F1 de tokens (0.250) en modo híbrido (solapamiento parcial), estándar en SQuAD.

Métricas Globales de Retrieval

Comparación Precision@k · Hit Rate@k · Recall@k · MRR · F1@k con k=5



Hit Rate@5

Híbrido lidera con 0.800 | TF-IDF 0.733

MRR

Híbrido 0.561 supera a TF-IDF 0.484

Recall@5

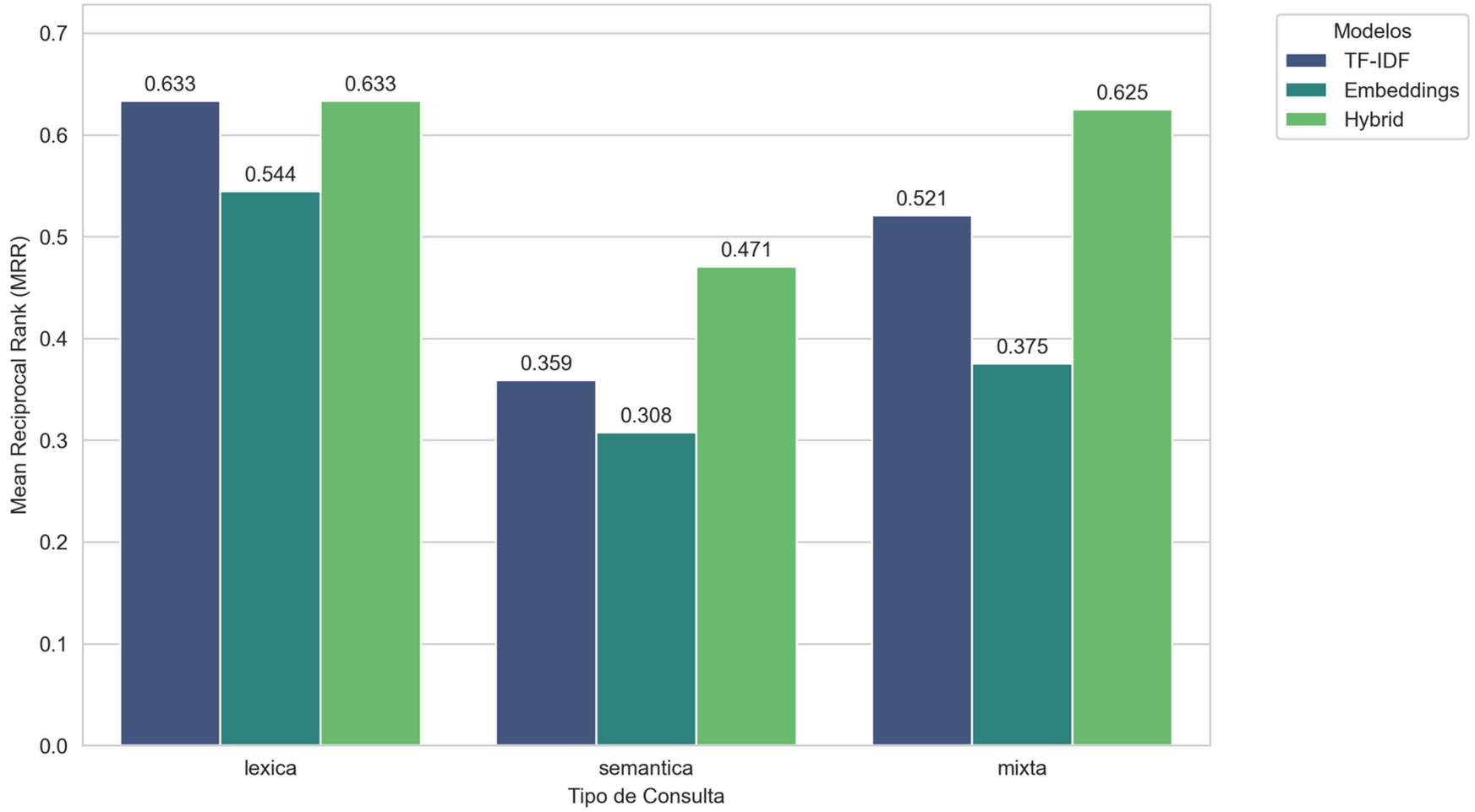
Híbrido 0.593 recupera más relevantes

F1@5

Híbrido 0.284 - mejor balance P/R

Evaluación Comparativa

¿Qué modelo funciona mejor y cuándo?



Léxicas

TF-IDF e Híbrido empatan - palabras exactas alcanza

Semánticas

Híbrido supera TF-IDF en +30% - embeddings agregan valor real

Mixtas

Embeddings e Híbrido lideran - señal semántica decide

MRR Global

Híbrido = 0.561 | TF-IDF = 0.484 | Embeddings = 0.397

QA F1: Híbrido extractivo = 0.250 (mejor) | EM = 0 en todos - esperable

API REST: El sistema completo, consultable

Request POST /consultar → Pipeline Singleton → Retriever Híbrido → QA Transformer → Response + doc_id

GET	/estado	Estado del pipeline: documentos, chunks y modelos cargados
POST	/buscar	Búsqueda con modelo elegido (tfidf / embeddings / hybrid)
POST	/comparar	Misma query en los 3 modelos, resultados lado a lado
POST	/consultar	Pipeline completo: retrieval + QA con trazabilidad
GET	/documento/{id}	Chunks de un documento específico por ID

api/pipeline.py (Singleton) · lazy loading del transformer · fallback controlado · docs en /docs

Demo en Vivo

Sistema corriendo en tiempo real

01

Levantar la API

```
uvicorn api.main:app --reload
```

→ *Abrir http://localhost:8000/docs*

02

POST /consultar

```
{ "query": "¿Qué es la brucelosis bovina?", "modelo": "hybrid", "modo_qa": "transformer",
"top_k": 5 }
```

→ *Query semántica: no usa palabras exactas del artículo*

03

Resultado con trazabilidad

```
"respuesta": "enfermedad producida por Brucella..." | "chunk_id": "Res_067-2019_sec3_sub0_c0" |
"score": 0.87
```

→ *Señalar: artículo 2 de Res 067/2019 — verificable en el PDF*

Limitaciones y Consideraciones Éticas

Limitaciones técnicas

01 Corpus de 10 resoluciones: suficiente para comparar modelos, insuficiente para cobertura operacional.

02 Embeddings zero-shot: sin fine-tuning en texto jurídico argentino.

03 FAISS en memoria: sin persistencia entre reinicios del servidor.

04 Gold standard con respuestas largas subestima el F1 del transformer.

Consideraciones éticas

Sesgo de corpus

Fuera del corpus, devuelve el chunk más parecido (puede ser incorrecto). El score bajo es señal de baja confianza.

Sesgo del modelo

MiniLM entrenado en corpus general, no en derecho argentino. Impacto bajo por vocabulario técnico.

Trazabilidad

Toda respuesta referencia doc_id y chunk_id verificable. Sin generación libre - el sistema no inventa.

Trabajo futuro: fine-tuning en corpus jurídico · LLM local para síntesis · persistencia FAISS · evaluación de equidad

CONCLUSIONES

01

Técnico

- Híbrido supera a TF-IDF y embeddings puros (MRR 0.561 vs 0.484 vs 0.397)
- Chunking estructural preserva coherencia semántica normativa
- Arquitectura RAG mitiga alucinaciones por diseño

02

Organizacional

- HR@5 = 0.800 → documento correcto en 80% de las búsquedas
- Trazabilidad completa: toda respuesta referencia su fuente
- API consultable en tiempo real, documentación automática

03

Académico

- Gold standard de 30 queries verificadas manualmente
- Pipeline reproducible: modo mock sin HuggingFace
- Decisiones de diseño justificadas con métricas

"El sistema no reemplaza al especialista - le da la respuesta correcta, con la fuente, en segundos."